Agentic AI

From concepts to building your own agent

A structured study guide • Notes compiled November 2026

What this guide covers

Part	Section	What you will learn
A	1. The three generations of AI	Traditional vs generative vs agentic AI
A	2. How an agent works	The perceive–reason–act–reflect loop
A	3. Six core capabilities	What makes a system 'agentic'
A	4. Single vs multi-agent	Architectures, patterns, when to use which
A	5. Use cases	Real-world examples of both
A	6. Challenges and risks	What can go wrong
B	7. The 10-step build roadmap	How to build an agentic system end to end
B	8. Code walkthrough	A complete minimal agent in Python
B	9. Frameworks and tools	LangGraph, CrewAI, AutoGen, SDKs
B	10. Learning path	What to practise, in order
–	Quick revision + glossary	One-page summary and key terms

How to use this guide

Part A builds understanding; Part B shows how to build. Highlighted boxes give the **key takeaways**. Use the last page for quick revision before interviews or exams.

PART A — UNDERSTANDING AGENTIC AI

1. The three generations of AI

Traditional AI (narrow / predictive AI) learns patterns from data to do one specific task, such as flagging spam, predicting loan default or recommending a product. It outputs a label, score or number. It does not create anything new and does not decide what to do next.

Generative AI (LLMs such as Claude or GPT; image models such as Stable Diffusion) learns the structure of language, images or code well enough to create new content. It is **reactive**: one prompt in, one response out. You drive every step.

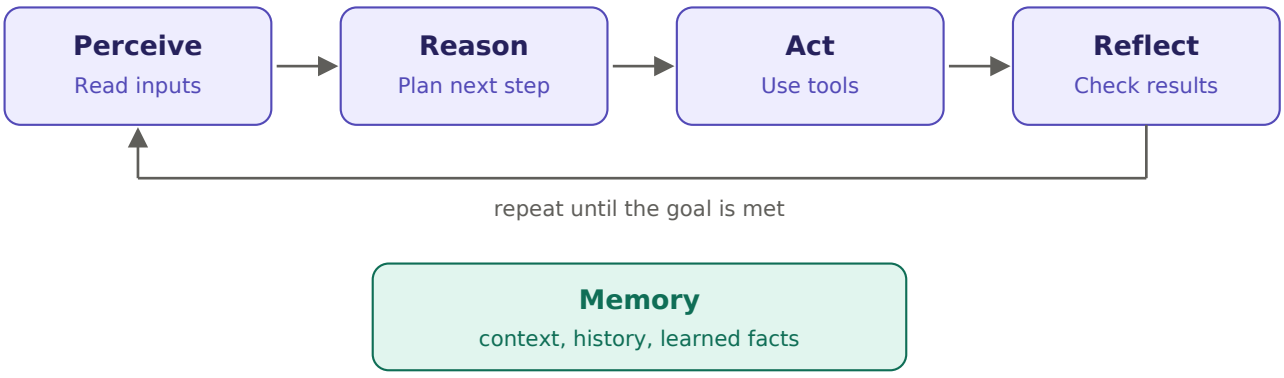
Agentic AI wraps a generative model inside a loop so it can pursue a goal. You give an objective (e.g. “book the cheapest flight to Delhi next Friday landing before 6 pm”), and it plans steps, uses tools (search, APIs, code, databases), checks results, corrects itself and continues until the goal is met or it needs your input. The LLM is the **brain**; the agent framework gives it **hands, memory and persistence**.

Aspect	Traditional AI	Generative AI	Agentic AI
Core job	Predict / classify	Create content	Achieve goals
Input → output	Data → label/score	Prompt → text, image, code	Goal → completed task
Autonomy	None	Low (one turn at a time)	High (multi-step, self-directed)
Uses tools?	No	Usually not	Yes — central to it
Memory	Training data only	Within the conversation	Short-term + long-term
Adapts mid-task?	No	Only if re-prompted	Yes, reacts to results
Example	Fraud detection model	Chatbot drafting an email	Agent that reads inbox, drafts replies, books meetings

Key takeaway
Traditional AI **analyses**. Generative AI **creates**. Agentic AI **acts**.

2. How an agent works: the agent loop

Every agentic system, however it is built, runs a version of this loop. Memory supports every step.



Stage	What happens	Example (flight booking)
Perceive	Reads the goal, user input, tool outputs, documents	Reads: 'cheapest flight to Delhi, Friday, land before 6 pm'
Reason	Decides the next best step	'First, search flights for Friday'
Act	Calls a tool or API	Calls flight-search API
Reflect	Checks if the result is good; adjusts	'3 results land after 6 pm — filter them out'
Repeat	Loops until the goal is achieved	Compares prices → books → confirms

3. Six core capabilities of an agent

#	Capability	What it means	Why it matters
1	Autonomy & goal orientation	You say what, not how. It chooses the steps itself.	No prompting at every turn
2	Planning & reasoning	Breaks big goals into sub-tasks. Patterns: ReAct, chain-of-thought, plan-and-execute.	Handles multi-step problems
3	Tool use	Web search, APIs, code execution, databases, email, browser control.	Lets AI **do**, not just talk
4	Memory	Short-term: current task context. Long-term: facts, preferences, past lessons (often a vector DB).	Continuity and improvement
5	Reflection & self-correction	Checks each result; retries, changes approach, or asks a human.	Fewer errors, more reliability
6	Perception of environment	Reads documents, screens, sensors, tool outputs and adapts.	Reacts to reality, not assumptions

Key takeaway
Without tools, an LLM can only talk. With tools, a loop, and memory, it becomes an agent.

4. Single-agent vs multi-agent systems

Single-agent system: one agent (one LLM brain) handles the whole task end to end with its tools. Analogy: a capable freelancer working alone.

Multi-agent system: several specialised agents, each with its own role, prompt, tools (sometimes its own model), collaborate. Analogy: a project team with a manager, researcher, writer and reviewer.

Common multi-agent patterns

Pattern	How it works	Example
Supervisor (hierarchical)	An orchestrator splits the job, delegates to workers, combines results	Manager agent assigns research to 3 workers

Pattern	How it works	Example
Sequential pipeline	Output of agent A feeds agent B, like an assembly line	Extract → verify → summarise
Collaborative / debate	Agents critique each other to improve quality	Coder agent + reviewer agent

Aspect	Single agent	Multi-agent
Structure	One agent does everything	Team of specialised agents
Best for	Well-defined, moderate tasks	Complex, multi-domain, long tasks
Cost & latency	Lower	Higher (many LLM calls)
Debugging	Easier	Harder (hand-offs, loops)
Quality on complex work	Can degrade (crowded context)	Usually better: specialisation + review
Parallel work	No	Yes

Rule of thumb
Start with a **single agent**. Move to multiple agents only when one agent clearly struggles: the task is too broad, the context too long, or you need independent checking.

5. Use cases

Single-agent examples

Use case	What the agent does
Customer support	Reads query → looks up order → checks refund policy → issues refund via API → replies
Personal assistant	'Move Thursday meetings to Friday' → reads calendar → finds slots → moves events → emails attendees
Coding agent	Reads bug report → explores codebase → edits files → runs tests → fixes until tests pass
Data analysis	Reads sales CSV → writes and runs Python → makes charts → explains why Q3 dropped

Multi-agent examples

Use case	Agents involved
Research report	Planner → parallel researchers (one per subtopic) → writer → fact-checker
Software team	Product manager → architect → coders → tester → code reviewer
Loan / insurance processing	Document extractor → identity & credit verifier → risk scorer → compliance checker → human approval
Supply chain	Demand forecaster ↔ inventory agent ↔ procurement agent (auto-reorder on demand spikes)

Use case	Agents involved
Travel planning	Flight agent + hotel agent + activities agent → coordinator builds itinerary within budget

6. Challenges and risks

Risk	What it means	Mitigation
Hallucinated actions	Agent acts on a wrong belief — worse than just saying something wrong	Verify with tools; validate inputs in code
Infinite loops / cost blow-up	Agent keeps retrying or wandering	Cap steps, time and spend
Prompt injection	Hidden instructions in web pages, emails or files hijack the agent	Treat external content as untrusted data
Over-permission	Agent can delete, pay or email without limits	Least privilege; human approval for high-stakes actions
Hard to debug	Many steps and hand-offs	Log every step; use tracing tools

PART B — BUILDING AN AGENTIC AI SYSTEM

7. The 10-step build roadmap

Building an agent is less about the model and more about the engineering around it: the loop, tools, memory and guardrails.

Step	What to do	Key output
1. Define goal & scope	One concrete job; inputs; what 'done' means; what it must never do	A written spec
2. Choose the brain (LLM)	Pick a model strong at tool use; optionally a cheaper model for simple sub-tasks	Model choice
3. Build the tools	Functions + clear name, description and input schema. Consider MCP connectors	Tool list
4. Write the agent loop	Send goal + tools → run requested tool → return result → repeat	Working agent
5. Add memory	Short-term: message history. Long-term: vector DB (RAG). Trim/summarise long context	Memory layer
6. Add planning & reflection	ReAct, plan-and-execute, self-critique — only if tests show a need	Better reliability
7. Go multi-agent (if needed)	Split into specialists under a supervisor	Agent team
8. Build guardrails	Least privilege, human approval, input validation, step/cost caps	Safe agent
9. Evaluate & observe	Log every step; 20–50 test tasks; track success, steps, cost	Metrics + traces
10. Deploy & iterate	API (e.g. FastAPI), background workers, DB for state, small rollout first	Production system

Before you start

Ask: do I really need an agent? If the steps are always the same, a fixed workflow (A → B → C) with one LLM call is cheaper and more reliable. Use an agent when the path varies depending on what it finds.

Step details

Step 1 — Define goal and scope. Narrow scope is the biggest predictor of success. Example: “Resolve refund requests under ₹5,000 for delivered orders; escalate anything else to a human.”

Step 2 — Choose the brain. Needs strong function calling and instruction following: Claude, GPT, Gemini, or open models such as Llama or Qwen if you must self-host.

Step 3 — Build the tools. Good tool design matters a lot:

- Clear names and precise descriptions (the model reads these to decide)
- Each tool does one focused thing
- Helpful error messages the agent can learn from
- Reuse existing **MCP** (Model Context Protocol) connectors for GitHub, Slack, Google Drive, etc.

Step 4 — Write the loop. Surprisingly small — see the full code in Section 8.

Step 5 — Add memory. The message list is short-term memory. Long-term memory is usually a vector database (Chroma, Pinecone, pgvector) searched by similarity — the same idea as RAG, used as a tool.

Step 6 — Planning and reflection. ReAct (thought → action → observation), plan-and-execute (write a plan, then revise), reflection (critique and fix before finishing).

Step 8 — Guardrails checklist:

- Minimum permissions per tool (read-only where possible)
- Human approval before payments, deletions or customer emails
- Validate tool inputs in code — never trust the model blindly
- Cap steps, time and spending
- Treat web pages, emails and documents as untrusted (prompt injection)

Step 9 — Evaluate. Trace with LangSmith, Langfuse or Arize Phoenix. Re-run your test set after every change to prompts, tools or model. Track: success rate, steps per task, cost per task, human-help rate.

8. Code walkthrough: a minimal agent in Python

This complete agent uses no framework, so every moving part is visible. It answers a customer question by deciding on its own to look up two orders, add the prices, and apply a discount.

Setup: `pip install anthropic`, set `ANTHROPIC_API_KEY`, then `python simple_agent.py`.

```
import json
import anthropic

client = anthropic.Anthropic()
MODEL = "claude-sonnet-5-5"
MAX_STEPS = 10  # guardrail: stop runaway loops


# ---------------------------------------------------------------
# 1. TOOLS: plain Python functions the agent is allowed to call
# ---------------------------------------------------------------
def calculator(expression: str) -> str:
    """Evaluate a simple arithmetic expression safely."""
    allowed = set("0123456789+-*/(). ")
    if not set(expression) <= allowed:
        return "Error: only numbers and + - * / ( ) are allowed."
    try:
        return str(eval(expression))
    except Exception as e:
        return f"Error: {e}"


# A fake "database" standing in for a real API or DB call
ORDERS = {
    "A101": {"item": "Headphones", "price": 2499, "status": "delivered"},
    "A102": {"item": "Keyboard", "price": 1799, "status": "in transit"},
}


def get_order(order_id: str) -> str:
    """Look up an order by its ID."""
    order = ORDERS.get(order_id)
    return json.dumps(order) if order else f"No order found with id {order_id}"


# Map tool names to functions
TOOL_FUNCTIONS = {"calculator": calculator, "get_order": get_order}

# Describe each tool to the model (name, purpose, input schema)
TOOLS = [
    {
        "name": "calculator",
        "description": "Evaluate an arithmetic expression, e.g. '2499 * 0.9'.",
        "input_schema": {
            "type": "object",
            "properties": {"expression": {"type": "string"}},
            "required": ["expression"],
        },
    },
    {
        "name": "get_order",
        "description": "Fetch details (item, price, status) of a customer order by ID.",
        "input_schema": {
            "type": "object",
            "properties": {"order_id": {"type": "string"}},
            "required": ["order_id"],
        },
    },
]

SYSTEM_PROMPT = (
    "You are a helpful customer-support agent. "
    "Use tools to look up facts instead of guessing. "
```

```
    "Think step by step, and when the task is complete, give a short final answer."
)


# ---------------------------------------------------------------
# 2. THE AGENT LOOP
# ---------------------------------------------------------------
def run_agent(goal: str) -> str:
    messages = [{"role": "user", "content": goal}]  # short-term memory

    for step in range(1, MAX_STEPS + 1):
        # REASON: ask the model what to do next
        response = client.messages.create(
            model=MODEL,
            max_tokens=1024,
            system=SYSTEM_PROMPT,
            tools=TOOLS,
            messages=messages,
        )
        messages.append({"role": "assistant", "content": response.content})

        # DONE: the model answered without asking for a tool
        if response.stop_reason != "tool_use":
            return "".join(b.text for b in response.content if b.type == "text")

        # ACT: run every tool the model asked for
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                func = TOOL_FUNCTIONS[block.name]
                result = func(**block.input)
                print(f"[step {step}] {block.name}({block.input}) -> {result}")
                tool_results.append(
                    {"type": "tool_result", "tool_use_id": block.id, "content": result}
                )

        # OBSERVE: feed results back so the model can reflect and continue
        messages.append({"role": "user", "content": tool_results})

    return "Stopped: reached the maximum number of steps."


if __name__ == "__main__":
    goal = (
        "Customer asks: what did I buy in orders A101 and A102, "
        "what's the total, and what is the total after a 10% discount?"
    )
    print("FINAL ANSWER:\n", run_agent(goal))
```

How the code maps to the agent loop

Code part	Agent concept
`calculator`, `get_order`	Tools — plain Python functions
`TOOLS` list	Tool descriptions + input schemas the model reads
`messages` list	Short-term memory
`client.messages.create(...)`	Reason — the model decides the next step
`stop_reason != "tool_use"`	Goal met — return final answer
Running `TOOL_FUNCTIONS[block.name]`	Act — execute the requested tool
Appending `tool_result`	Observe / reflect — model sees results and continues
`MAX_STEPS`	Guardrail against runaway loops

Try this next
Add a fake **issue_refund** tool that asks for human confirmation before running. Official tool-use docs: docs.claude.com/en/docs/agents-and-tools/tool-use/overview

9. Frameworks and tools

Framework / tool	Best for	Style
LangGraph	Control and reliability in production	Agents as a graph of steps with explicit state
CrewAI	Quick start with role-based teams	Define agents by role: 'researcher', 'writer'
AutoGen	Agents that converse with each other	Multi-agent conversations
OpenAI Agents SDK / Claude Agent SDK	Lightweight building blocks	Tools, hand-offs, tracing
MCP (Model Context Protocol)	Plugging in tools/data sources	Standard connector protocol
Chroma / Pinecone / pgvector	Long-term memory	Vector databases
LangSmith / Langfuse / Phoenix	Observability & evaluation	Tracing and test runs
FastAPI + task queue	Deployment	API + background workers

Advice
Learn the raw loop first (Section 8), then pick a framework — you will understand what it does for you.

10. Learning path

Stage	Practice task	Skill gained
1	Run and modify simple_agent.py; add an issue_refund tool with confirmation	Agent loop, tool design, guardrails
2	Add a web search tool and a small vector-DB memory	Real tools, RAG, long-term memory
3	Rebuild the same agent in LangGraph	Framework structure, state
4	Build a two-agent system (researcher + writer)	Multi-agent coordination
5	Add logging, a test set and guardrails	Evaluation and production readiness

Quick revision

Concept	Remember it as
Traditional AI	Analyses — predicts a label or score
Generative AI	Creates — one prompt in, one response out
Agentic AI	Acts — pursues a goal in a loop with tools and memory
Agent loop	Perceive → Reason → Act → Reflect → repeat
6 capabilities	Autonomy, planning, tool use, memory, reflection, perception
Single agent	One freelancer — simple, cheap, easy to debug
Multi-agent	A team — specialised, parallel, higher quality but costlier
Multi-agent patterns	Supervisor, sequential pipeline, collaborative/debate
Build order	Scope → LLM → tools → loop → memory → planning → multi-agent → guardrails → evaluate → deploy
Golden rules	Start simple; single agent first; least privilege; human approval for risky actions; log everything

Glossary

Term	Meaning
LLM	Large language model — the reasoning 'brain' of an agent
Tool / function calling	The model requests a function to be run with specific inputs
ReAct	Reason + Act: alternating thoughts, actions and observations
RAG	Retrieval-augmented generation — fetching relevant knowledge before answering
Vector database	Stores text as embeddings for similarity search; used for long-term memory
MCP	Model Context Protocol — a standard way to connect tools and data to AI agents
Orchestrator / supervisor	Agent that splits work and delegates to other agents
Prompt injection	Malicious instructions hidden in content the agent reads
Guardrails	Limits and checks that keep an agent safe (permissions, approvals, caps)
Context window	How much text the model can consider at once